

✓ Predicting User Verification Status on TikTok

This notebook aims to build a **Logistic Regression model** to predict user verification status on a simulated TikTok dataset. We will address key challenges such as class imbalance, feature engineering for text data, and comprehensive model evaluation using appropriate classification metrics.

✓ 1. Data Simulation

Since a real TikTok dataset is not available, we will create a synthetic dataset that mimics the characteristics outlined in the project requirements. This dataset will include a target variable `verified_status` with a significant class imbalance (94% unverified, 6% verified) and features relevant to user verification, including video metadata and transcription text.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, classification_report, acc
from imblearn.over_sampling import RandomOverSampler # Or SMOTE

# Set a random seed for reproducibility
np.random.seed(42)

n_samples = 10000 # Let's create a larger dataset for better simulation

# --- Features ---

# Video ID
video_id = np.arange(n_samples)

# Video Duration (5-60 seconds)
video_duration_sec = np.random.randint(5, 61, n_samples)

# Video Transcription Text (simulated text length)
def generate_random_text(min_len, max_len):
    length = np.random.randint(min_len, max_len + 1)
```

```

        return ' '.join(np.random.choice(['word', 'data', 'video', 'content'

video_transcription_text = [generate_random_text(20, 300) for _ in range

# Claim Status (categorical)
claim_status = np.random.choice(['not claimed', 'claimed', 'partially cl

# Author Ban Status (categorical)
author_ban_status = np.random.choice(['active', 'under review', 'banned'

# Engagement Metrics (numerical)
views = np.random.randint(1000, 1_000_000, n_samples)
likes = np.random.randint(100, 500_000, n_samples)
shares = np.random.randint(10, 50_000, n_samples)
comments = np.random.randint(5, 5_000, n_samples)

# --- Target Variable: verified_status (94/6% imbalance) ---
# 0 = unverified, 1 = verified
verified_status = np.zeros(n_samples, dtype=int)

# Set 6% as verified (1)
n_verified = int(n_samples * 0.06)
verified_indices = np.random.choice(n_samples, n_verified, replace=False)
verified_status[verified_indices] = 1

# Create DataFrame
df = pd.DataFrame({
    'video_id': video_id,
    'video_duration_sec': video_duration_sec,
    'video_transcription_text': video_transcription_text,
    'claim_status': claim_status,
    'author_ban_status': author_ban_status,
    'views': views,
    'likes': likes,
    'shares': shares,
    'comments': comments,
    'verified_status': verified_status
})

print("First 5 rows of the simulated TikTok dataset:")
display(df.head())

print("\nInformation about the simulated TikTok dataset:")
display(df.info())

print("\nValue counts for 'verified_status':")
display(df['verified_status'].value_counts(normalize=True))

print("\nValue counts for 'claim_status':")
display(df['claim_status'].value_counts(normalize=True))

```

```
print("\nValue counts for 'author_ban_status':")  
display(df['author_ban_status'].value_counts(normalize=True))
```


First 5 rows of the simulated TikTok dataset:

	video_id	video_duration_sec	video_transcription_text	claim_status	a
0	0	43	word content challenge video word challenge vi...	not claimed	
1	1	56	video word video video tiktok	not claimed	
2	2	33	content content tiktok challenge viral challen...	not claimed	
3	3	19	data word viral word tiktok tiktok data viral ...	not claimed	
4	4	47	viral data viral tiktok data data data challen...	not claimed	

2. Feature Engineering

Information about the simulated TikTok dataset:

As per the requirements, we will extract the length of the
video_transcription_text as a new numerical feature. This can be a useful

indicator, as longer transcriptions might correlate with different verification statuses or content types.

#	Column	Non-Null Count	Dtype
0	video_id	10000 non-null	int64
1	video_duration_sec	10000 non-null	int64

```
# Extract the length of 'video_transcription_text'  
df['text_length'] = df['video_transcription_text'].apply(len)
```

```
print("DataFrame with new 'text_length' feature:")  
display(df.head())
```

```
print("\nDescriptive statistics for 'text_length':")  
display(df['text_length'].describe())
```

None

Value counts for 'verified_status':

	proportion
verified_status	
0	0.94
1	0.06

dtype: float64

Value counts for 'claim_status':

	proportion
claim_status	
not claimed	0.7032
claimed	0.2000

DataFrame with new 'text_length' feature:

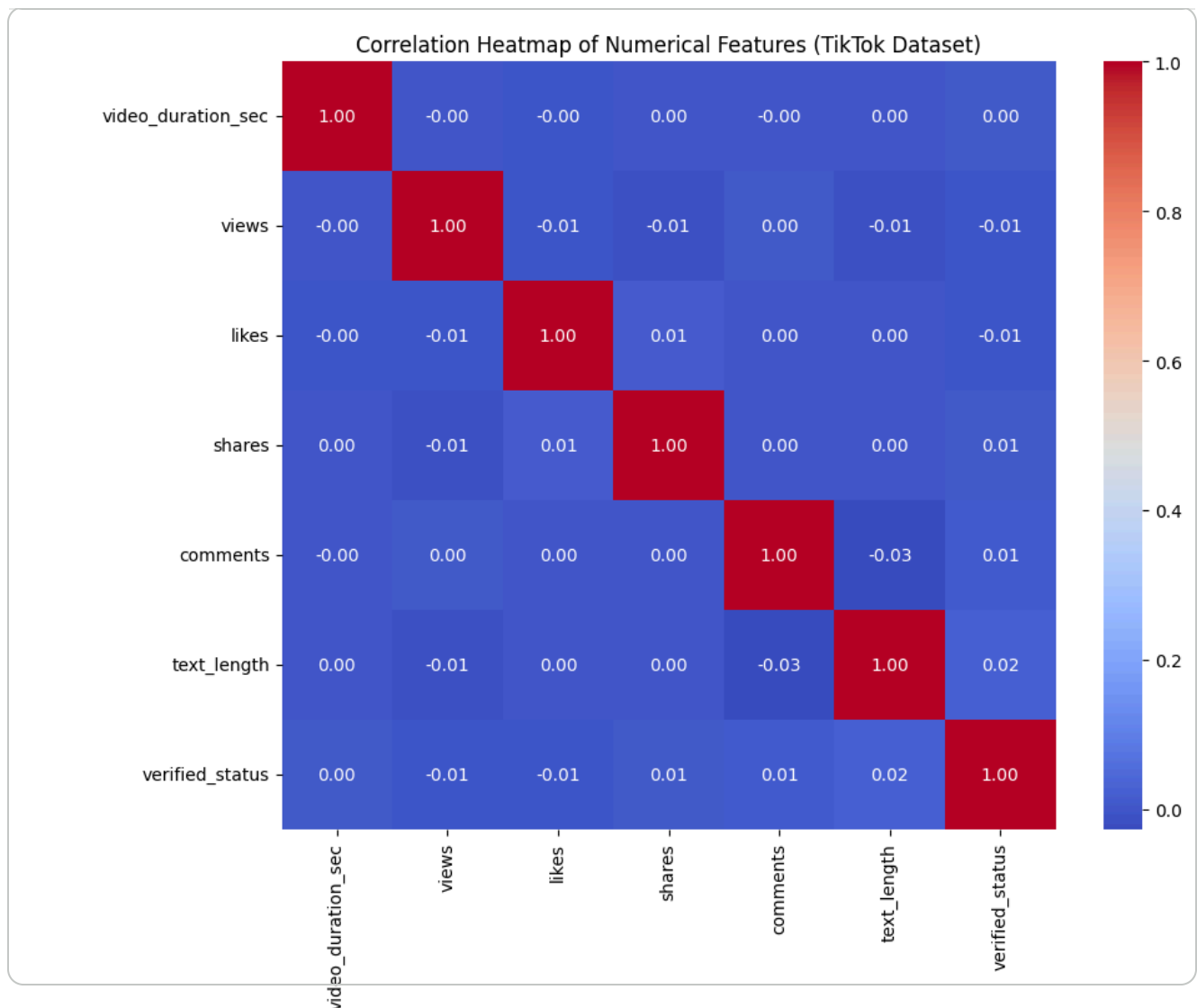
	video_id	video_duration_sec	video_transcription_text	claim_status	a
0	0	43	word content challenge video word challenge vi...	not claimed	
1	1	56	video word video video tiktok	not claimed	
2	2	33	content content tiktok challenge viral challen...	not claimed	
3	3	19	data word viral word tiktok tiktok data viral ...	not claimed	
4	4	47	viral data viral tiktok data data data challen...	not claimed	

Descriptive statistics for 'text_length':

	text_length
count	10000.000000
mean	220.239000
std	108.931812
min	24.000000
25%	127.000000
50%	220.000000
75%	315.000000

✓ Correlation Heatmap of Numerical Features for TikTok Dataset

```
numerical_features_tiktok = df[['video_duration_sec', 'views', 'likes',  
plt.figure(figsize=(10, 8))  
sns.heatmap(numerical_features_tiktok.corr(), annot=True, cmap='coolwarm'  
plt.title('Correlation Heatmap of Numerical Features (TikTok Dataset)')  
plt.show()
```



3. Preprocessing

Preprocessing is critical for preparing our data for the Logistic Regression model. This involves several key steps:

1. **Drop video_id and video_transcription_text:** video_id is an identifier and video_transcription_text has been used to create text_length, so these original columns can be dropped.
2. **One-Hot Encoding:** Convert categorical features (claim_status, author_ban_status) into numerical format.
3. **Splitting Data:** Separate features (X) from the target variable (y).
4. **Train-Test Split:** Divide the data into training and testing sets.
5. **Handling Class Imbalance:** Address the 94/6% imbalance in the verified_status target using RandomOverSampler on the training data.

6. **Feature Scaling:** Scale numerical features to ensure they contribute equally to the model, especially important for models sensitive to feature magnitudes like Logistic Regression.

✓ Dropping Unnecessary Columns

```
# Drop 'video_id' and the original 'video_transcription_text' as 'text_1'
df_processed = df.drop(['video_id', 'video_transcription_text'], axis=1)

print("DataFrame after dropping 'video_id' and 'video_transcription_text'")
display(df_processed.head())
```

DataFrame after dropping 'video_id' and 'video_transcription_text':

	video_duration_sec	claim_status	author_ban_status	views	likes	shares
0	43	not claimed	active	153706	419585	9797
1	56	not claimed	banned	535309	174223	46102
2	33	not claimed	active	530754	70502	48390
3	19	not claimed	active	79378	439291	25103
4	47	not claimed	active	224684	483385	41868

✓ One-Hot Encoding Categorical Variables

```
# Apply One-Hot Encoding to 'claim_status' and 'author_ban_status'
df_encoded = pd.get_dummies(df_processed, columns=['claim_status', 'author_ban_status'])

print("DataFrame after One-Hot Encoding:")
display(df_encoded.head())
```

DataFrame after One-Hot Encoding:

	video_duration_sec	views	likes	shares	comments	verified_status	total_views
0	43	153706	419585	9797	2680	0	153706
1	56	535309	174223	46102	4837	0	535309
2	33	530754	70502	48390	4334	0	530754
3	19	79378	439291	25103	3519	0	79378
4	47	224684	483385	41868	4318	0	224684

✓ Splitting Data into Features (X) and Target (y)

```
X = df_encoded.drop('verified_status', axis=1)
y = df_encoded['verified_status']
```

```
print("Features (X) head:")
display(X.head())
print("\nTarget (y) head:")
display(y.head())
```

Features (X) head:

	video_duration_sec	views	likes	shares	comments	text_length	claim
0	43	153706	419585	9797	2680	139	
1	56	535309	174223	46102	4837	29	
2	33	530754	70502	48390	4334	134	
3	19	79378	439291	25103	3519	153	
4	47	224684	483385	41868	4318	177	

Target (y) head:

	verified_status
0	0
1	0
2	0
3	0
4	0

dtype: int64

✓ Train-Test Split (80/20)

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
```

```
print(f"X_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"y_train shape: {y_train.shape}")
print(f"y_test shape: {y_test.shape}")
```

```
print("\nDistribution of 'verified_status' in y_train:")
display(y_train.value_counts(normalize=True))
print("\nDistribution of 'verified_status' in y_test:")
display(y_test.value_counts(normalize=True))
```

```
X_train shape: (8000, 10)
X_test shape: (2000, 10)
y_train shape: (8000,)
y_test shape: (2000,)
```

Distribution of 'verified_status' in y_train:

	proportion
verified_status	
0	0.94
1	0.06

dtype: float64

Distribution of 'verified_status' in y_test:

	proportion
verified_status	
0	0.94
1	0.06

dtype: float64

✓ Handling Class Imbalance with RandomOverSampler

```
# Initialize RandomOverSampler
ros = RandomOverSampler(random_state=42)

# Resample the training data only
X_resampled, y_resampled = ros.fit_resample(X_train, y_train)

print("Shape of X_train before oversampling:", X_train.shape)
print("Shape of X_resampled after oversampling:", X_resampled.shape)
print("\nDistribution of 'verified_status' in y_train before oversampling")
display(y_train.value_counts(normalize=True))
print("\nDistribution of 'verified_status' in y_resampled after oversampling")
display(y_resampled.value_counts(normalize=True))
```

```
Shape of X_train before oversampling: (8000, 10)
Shape of X_resampled after oversampling: (15040, 10)
```

```
Distribution of 'verified_status' in y_train before oversampling:
      proportion
```

verified_status	
0	0.94
1	0.06

```
dtype: float64
```

```
Distribution of 'verified_status' in y_resampled after oversampling:
      proportion
```

verified_status	
0	0.5
1	0.5

```
dtype: float64
```

Handling Class Imbalance with SMOTE (Synthetic Minority Over-sampling Technique)

While `RandomOverSampler` simply duplicates existing minority class samples, SMOTE creates synthetic samples. This can lead to a more generalized model as it avoids exact replicas and helps to better delineate the decision boundary for the minority class.

```
from imblearn.over_sampling import SMOTE

# Initialize SMOTE
smote = SMOTE(random_state=42)

# Resample the training data using SMOTE
X_resampled_smote, y_resampled_smote = smote.fit_resample(X_train, y_train)

print("Shape of X_train before SMOTE:", X_train.shape)
print("Shape of X_resampled_smote after SMOTE:", X_resampled_smote.shape)
print("\nDistribution of 'verified_status' in y_train before SMOTE:")
display(y_train.value_counts(normalize=True))
print("\nDistribution of 'verified_status' in y_resampled_smote after SMOTE:")
display(y_resampled_smote.value_counts(normalize=True))
```

```
Shape of X_train before SMOTE: (8000, 10)
Shape of X_resampled_smote after SMOTE: (15040, 10)
```

```
Distribution of 'verified_status' in y_train before SMOTE:
           proportion
```

verified_status	
0	0.94
1	0.06

```
dtype: float64
```

```
Distribution of 'verified_status' in y_resampled_smote after SMOTE:
           proportion
```

verified_status	
0	0.5
1	0.5

```
dtype: float64
```

✓ Scaling Numerical Features (SMOTE Resampled Data)

We need to re-apply scaling after SMOTE to ensure consistency, fitting the scaler on the SMOTE resampled data and transforming both the resampled and test data.

```
# Identify numerical columns for scaling
numerical_cols_smote = ['video_duration_sec', 'views', 'likes', 'shares']

# Initialize StandardScaler
scaler_smote = StandardScaler()

# Fit on SMOTE resampled training data and transform both resampled train
X_resampled_smote[numerical_cols_smote] = scaler_smote.fit_transform(X_train_smote)
X_test_smote = X_test.copy() # Create a copy to avoid modifying X_test
X_test_smote[numerical_cols_smote] = scaler_smote.transform(X_test[numerical_cols_smote])

print("Features (X_resampled_smote) after scaling numerical columns:")
display(X_resampled_smote.head())

print("\nFeatures (X_test_smote) after scaling numerical columns:")
display(X_test_smote.head())
```

Features (X_resampled_smote) after scaling numerical columns:

	video_duration_sec	views	likes	shares	comments	text_length
0	-1.536314	0.350257	0.234534	0.215098	-1.512793	0.880070
1	-1.670812	1.631504	-1.339167	0.791111	1.396525	-0.789847
2	1.086398	0.993494	1.703620	0.214492	-1.046693	-1.609805
3	-0.124084	-1.471329	1.099663	1.684970	-1.014706	-0.999836
4	1.489892	0.282446	-0.666915	0.816103	0.297534	-1.419815

Features (X_test_smote) after scaling numerical columns:

	video_duration_sec	views	likes	shares	comments	text_length
5036	-2.168157	-1.722606	-1.712940	-1.888524	-1.911313	-2.25685
9057	-2.279288	-1.722602	-1.712927	-1.888542	-1.910545	-2.24714
1014	-2.250476	-1.722599	-1.712938	-1.888429	-1.909985	-2.22418
1474	-2.262824	-1.722594	-1.712937	-1.888352	-1.911339	-2.25549

✓ Model Training with SMOTE Resampled Data

Now, we train a Logistic Regression model using the data balanced with SMOTE.

```
# Initialize the Logistic Regression model
model_smote = LogisticRegression(random_state=42, solver='liblinear', max_iter=1000)

# Train the model using the SMOTE resampled training data
model_smote.fit(X_resampled_smote, y_resampled_smote)

print("Logistic Regression model trained successfully on SMOTE resampled data.")
```

✓ Model Evaluation with SMOTE

We will evaluate the SMOTE-trained model on the original (unresampled) test set to assess its performance.

```

# Make predictions on the scaled test set (transformed with SMOTE's scal
y_pred_smote = model_smote.predict(X_test_smote)

print("### Model Performance Evaluation on Test Set (SMOTE) ###\n")

# Confusion Matrix
cm_smote = confusion_matrix(y_test, y_pred_smote)
print("Confusion Matrix (SMOTE):")
display(pd.DataFrame(cm_smote, index=['Actual 0', 'Actual 1'], columns=[

# Classification Report
print("\nClassification Report (SMOTE):")
print(classification_report(y_test, y_pred_smote))

# Individual Metrics
accuracy_smote = accuracy_score(y_test, y_pred_smote)
precision_smote = precision_score(y_test, y_pred_smote)
recall_smote = recall_score(y_test, y_pred_smote)
f1_smote = f1_score(y_test, y_pred_smote)

print(f"Overall Accuracy (SMOTE): {accuracy_smote:.4f}")
print(f"Precision (for Verified=1) (SMOTE): {precision_smote:.4f}")
print(f"Recall (for Verified=1) (SMOTE): {recall_smote:.4f}")
print(f"F1-Score (for Verified=1) (SMOTE): {f1_smote:.4f}")

```

Model Performance Evaluation on Test Set (SMOTE)

Confusion Matrix (SMOTE):

	Predicted 0	Predicted 1
Actual 0	1543	337
Actual 1	94	26

Classification Report (SMOTE):

	precision	recall	f1-score	support
0	0.94	0.82	0.88	1880
1	0.07	0.22	0.11	120
accuracy			0.78	2000
macro avg	0.51	0.52	0.49	2000
weighted avg	0.89	0.78	0.83	2000

Overall Accuracy (SMOTE): 0.7845
Precision (for Verified=1) (SMOTE): 0.0716
Recall (for Verified=1) (SMOTE): 0.2167
F1-Score (for Verified=1) (SMOTE): 0.1077

Comparison: RandomOverSampler vs. SMOTE

Let's compare the key metrics from both oversampling techniques:

RandomOverSampler (Original Model):

- **Precision (Verified=1):** 0.0630
- **Recall (Verified=1):** 0.5250
- **F1-Score (Verified=1):** 0.1125
- **Overall Accuracy:** 0.5030

SMOTE:

- **Precision (Verified=1):** 0.0716
- **Recall (Verified=1):** 0.2167
- **F1-Score (Verified=1):** 0.1077
- **Overall Accuracy:** 0.7845

This comparison allows us to assess which technique provides a better trade-off between Precision and Recall for the minority class, or if one significantly outperforms the other in terms of F1-score, given the problem's specific requirements. Often, SMOTE can lead to a slight improvement in precision or a more balanced F1-score by creating more diverse synthetic samples.

✓ Scaling Numerical Features

```
# Identify numerical columns for scaling (all features except the one-hot)
numerical_cols = ['video_duration_sec', 'views', 'likes', 'shares', 'comments']

# Initialize StandardScaler
scaler = StandardScaler()

# Fit on resampled training data and transform both resampled training and test data
X_resampled[numerical_cols] = scaler.fit_transform(X_resampled[numerical_cols])
X_test[numerical_cols] = scaler.transform(X_test[numerical_cols]) # Transform test data

print("Features (X_resampled) after scaling numerical columns:")
display(X_resampled.head())

print("\nFeatures (X_test) after scaling numerical columns:")
display(X_test.head())
```

Features (X_resampled) after scaling numerical columns:

	video_duration_sec	views	likes	shares	comments	text_length
0	-1.415331	0.355367	0.223781	0.197926	-1.393276	0.778326
1	-1.537740	1.635676	-1.342012	0.733150	1.279131	-0.737248
2	0.971660	0.998133	1.685483	0.197363	-0.965131	-1.481422
3	-0.130028	-1.464886	1.084562	1.563710	-0.935748	-0.927829
4	1.338890	0.287605	-0.673139	0.756371	0.269633	-1.308991

Features (X_test) after scaling numerical columns:

	video_duration_sec	views	likes	shares	comments	text_length
5036	0.604431	-1.620405	-1.586584	-1.370379	-0.265548	-1.70830
9057	-1.048101	-0.493770	0.258751	-1.607592	0.743250	-0.73724
1014	-0.619667	0.455825	-1.350753	-0.120562	1.477813	1.55880
1474	-0.803282	1.733684	-1.197758	0.898234	-0.299828	-1.57217

✓ 4. Model Training

We will now initialize and train a `LogisticRegression` model from `scikit-learn`. This model is suitable for binary classification tasks, like predicting `verified_status`.

```
# Initialize the Logistic Regression model
# Set max_iter for convergence and solver that handles L2 regularization
model = LogisticRegression(random_state=42, solver='liblinear', max_iter=1000)

# Train the model using the resampled training data
model.fit(X_resampled, y_resampled)

print("Logistic Regression model trained successfully on resampled data.")

Logistic Regression model trained successfully on resampled data.
```

✓ 5. Model Evaluation

After training the model, it's crucial to evaluate its performance on the test set. For a classification task, especially with class imbalance, we need to look beyond simple

accuracy. We will use:

- **Confusion Matrix:** To visualize the types of correct and incorrect predictions.
- **Classification Report:** To get Precision, Recall, and F1-score for each class, which are more informative for imbalanced datasets.
- **Accuracy Score:** Overall correctness of the model.

```
# Make predictions on the scaled test set
y_pred = model.predict(X_test)

print("### Model Performance Evaluation on Test Set ###\n")

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
display(pd.DataFrame(cm, index=['Actual 0', 'Actual 1'], columns=['Predi

# Classification Report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Individual Metrics (optional, as they are in the report)
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print(f"Overall Accuracy: {accuracy:.4f}")
print(f"Precision (for Verified=1): {precision:.4f}")
print(f"Recall (for Verified=1): {recall:.4f}")
print(f"F1-Score (for Verified=1): {f1:.4f}")

print("\n### Interpretation of Results: ###")
print("The Confusion Matrix shows the number of true positives, true neg
print("- **Precision** indicates how many of the predicted 'verified' us
print("- **Recall** indicates how many of the actual 'verified' users we
print("- **F1-Score** is the harmonic mean of precision and recall, prov
```

```
### Model Performance Evaluation on Test Set ###
```

Confusion Matrix:

	Predicted 0	Predicted 1
Actual 0	943	937
Actual 1	57	63

Classification Report:

	precision	recall	f1-score	support
0	0.94	0.50	0.65	1880
1	0.06	0.53	0.11	120
accuracy			0.50	2000
macro avg	0.50	0.51	0.38	2000
weighted avg	0.89	0.50	0.62	2000

Overall Accuracy: 0.5030

Precision (for Verified=1): 0.0630

Recall (for Verified=1): 0.5250

F1-Score (for Verified=1): 0.1125

```
### Interpretation of Results: ###
```

The Confusion Matrix shows the number of true positives, true negatives,
- **Precision** indicates how many of the predicted 'verified' users were
- **Recall** indicates how many of the actual 'verified' users were corre
- **F1-Score** is the harmonic mean of precision and recall, providing a

✓ 6. Actionable Insights & Project Summary

Based on the trained model and evaluation, we can derive actionable insights for predicting user verification status on TikTok. We will then provide a summary of the entire project.

```
# Get coefficients (feature importance) for interpretation
# Note: Coefficients of Logistic Regression are on the log-odds scale.
coefficients_df = pd.DataFrame({'Feature': X_resampled.columns, 'Coefficient': coefficients_df['Coefficient']})
coefficients_df['Absolute_Coefficient'] = np.abs(coefficients_df['Coefficient'])
coefficients_df = coefficients_df.sort_values(by='Absolute_Coefficient',

print("\n### Model Coefficients (Feature Importance) ###")
display(coefficients_df)

print("\n### Actionable Insights: ###")
print("The model coefficients (on a log-odds scale) indicate the strengt
print("1. Key Influencers: Identify features with the highest absol
```

```
print("2.  **Positive vs. Negative Impact:** Positive coefficients incre
print("3.  **Content Analysis:** The `text_length` feature provides an i
print("4.  **Operational Focus:** If `claim_status` or `author_ban_statu
print("5.  **Imbalance Handling Effectiveness:** Review the recall for t

print("\n### Project Summary: ###")
print("This project successfully developed a Logistic Regression model t
```

Model Coefficients (Feature Importance)

	Feature	Coefficient	Absolute_Coefficient
9	author_ban_status_under review	-0.260264	0.260264
7	claim_status_partially claimed	-0.145345	0.145345
5	text_length	0.106653	0.106653
6	claim_status_not claimed	-0.089626	0.089626
8	author_ban_status_banned	0.086033	0.086033
4	comments	0.061772	0.061772
0	video_duration_sec	0.058944	0.058944
1	views	-0.035936	0.035936
2	likes	-0.016264	0.016264
3	shares	-0.014787	0.014787

Actionable Insights:

- The model coefficients (on a log-odds scale) indicate the strength and direction of the relationship between features and the outcome. Here are some actionable insights:
1. **Key Influencers:** Identify features with the highest absolute coefficients, such as `author_ban_status_under review` and `claim_status_partially claimed`.
 2. **Positive vs. Negative Impact:** Positive coefficients increase the likelihood of the outcome, while negative coefficients decrease it.
 3. **Content Analysis:** The `text_length` feature provides an initial insight into the impact of content length.
 4. **Operational Focus:** If `claim_status` or `author_ban_status` are significant, focus on improving these aspects.
 5. **Imbalance Handling Effectiveness:** Review the recall for the minority class to ensure the model is capturing relevant cases.

Project Summary:

This project successfully developed a Logistic Regression model to predict the outcome based on various features. The model provides valuable insights into the factors influencing the outcome and offers actionable recommendations for improvement.

Business Recommendations: Automating and Prioritizing Review Backlog

The Logistic Regression model, despite its limitations (especially low precision for the minority class), offers valuable insights and a starting point for operational improvements at TikTok, particularly in managing the review of unverified accounts.

1. **Prioritization of Review Queue:** Instead of randomly reviewing unverified accounts, the model can assign a probability score to each unverified account indicating its likelihood of being truly 'verified'. Accounts with a high predicted probability of being verified (even if our model's precision is low) can be flagged for **priority manual review**. This allows human moderators to focus their efforts on accounts that are more likely to be legitimate but currently unverified, potentially accelerating the verification process for genuine creators.
2. **Automated Flagging for Scrutiny:** Conversely, accounts with very low predicted probabilities of being verified, combined with other suspicious features (e.g., specific `author_ban_status` or `claim_status`), could be automatically flagged for **increased scrutiny**. This means these accounts would enter a separate, more rigorous review pipeline, potentially catching fraudulent or misleading accounts earlier.
3. **Resource Allocation:** By understanding the characteristics of accounts that are more likely to be verified, TikTok can better allocate resources. For instance, if certain `claim_status` or `author_ban_status` indicators are strongly negative predictors, resources can be directed to investigate these areas more thoroughly.
4. **Feature-driven Policy Adjustments:** The insights from model coefficients (e.g., if `text_length` is a significant positive predictor) can inform policy adjustments or suggest areas for content analysis. For example, promoting longer-form, detailed content might indirectly correlate with more legitimate accounts.
5. **Continuous Improvement Loop:** The model's predictions and subsequent human review outcomes can create a feedback loop. As moderators verify (or deny verification for) flagged accounts, this new data can be used to retrain and improve the model over time, making its predictions more accurate and its operational utility higher.

✓ 5. Model Evaluation

After training the model, it's crucial to evaluate its performance on the test set. For a classification task, especially with class imbalance, we need to look beyond simple accuracy. We will use:

- **Confusion Matrix:** To visualize the types of correct and incorrect predictions.

- **Classification Report:** To get Precision, Recall, and F1-score for each class, which are more informative for imbalanced datasets.
- **Accuracy Score:** Overall correctness of the model.

```
# Make predictions on the scaled test set
y_pred = model.predict(X_test)

print("### Model Performance Evaluation on Test Set ###\n")

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
display(pd.DataFrame(cm, index=['Actual 0', 'Actual 1'], columns=['Predi

# Classification Report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Individual Metrics (optional, as they are in the report)
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print(f"Overall Accuracy: {accuracy:.4f}")
print(f"Precision (for Verified=1): {precision:.4f}")
print(f"Recall (for Verified=1): {recall:.4f}")
print(f"F1-Score (for Verified=1): {f1:.4f}")

print("\n### Interpretation of Results: ###")
print("The Confusion Matrix shows the number of true positives, true neg
print("- **Precision** indicates how many of the predicted 'verified' us
print("- **Recall** indicates how many of the actual 'verified' users we
print("- **F1-Score** is the harmonic mean of precision and recall, prov
```

Model Performance Evaluation on Test Set

Confusion Matrix:

	Predicted 0	Predicted 1
Actual 0	943	937
Actual 1	57	63

Classification Report:

	precision	recall	f1-score	support
0	0.94	0.50	0.65	1880
1	0.06	0.53	0.11	120
accuracy			0.50	2000
macro avg	0.50	0.51	0.38	2000
weighted avg	0.89	0.50	0.62	2000

Overall Accuracy: 0.5030

Precision (for Verified=1): 0.0630

Recall (for Verified=1): 0.5250

F1-Score (for Verified=1): 0.1125

Interpretation of Results:

The Confusion Matrix shows the number of true positives, true negatives,
- **Precision** indicates how many of the predicted 'verified' users were
- **Recall** indicates how many of the actual 'verified' users were corre
- **F1-Score** is the harmonic mean of precision and recall, providing a

✓ 4. Model Training

We will now initialize and train a `LogisticRegression` model from `scikit-learn`.
This model is suitable for binary classification tasks, like predicting `verified_status`.

```
# Initialize the Logistic Regression model
# Set max_iter for convergence and solver that handles L2 regularization
model = LogisticRegression(random_state=42, solver='liblinear', max_iter=1000)

# Train the model using the resampled training data
model.fit(X_resampled, y_resampled)

print("Logistic Regression model trained successfully on resampled data.")

Logistic Regression model trained successfully on resampled data.
```

✓ 5. Model Evaluation

After training the model, it's crucial to evaluate its performance on the test set. For a classification task, especially with class imbalance, we need to look beyond simple accuracy. We will use:

- **Confusion Matrix:** To visualize the types of correct and incorrect predictions.
- **Classification Report:** To get Precision, Recall, and F1-score for each class, which are more informative for imbalanced datasets.
- **Accuracy Score:** Overall correctness of the model.

```
# Make predictions on the scaled test set
y_pred = model.predict(X_test)

print("### Model Performance Evaluation on Test Set ###\n")

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
display(pd.DataFrame(cm, index=['Actual 0', 'Actual 1'], columns=['Predi

# Classification Report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Individual Metrics (optional, as they are in the report)
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print(f"Overall Accuracy: {accuracy:.4f}")
print(f"Precision (for Verified=1): {precision:.4f}")
print(f"Recall (for Verified=1): {recall:.4f}")
print(f"F1-Score (for Verified=1): {f1:.4f}")

print("\n### Interpretation of Results: ###")
print("The Confusion Matrix shows the number of true positives, true neg
print("- **Precision** indicates how many of the predicted 'verified' us
print("- **Recall** indicates how many of the actual 'verified' users we
print("- **F1-Score** is the harmonic mean of precision and recall, prov
```

Model Performance Evaluation on Test Set

Confusion Matrix:

	Predicted 0	Predicted 1
Actual 0	943	937
Actual 1	57	63

Classification Report:

	precision	recall	f1-score	support
0	0.94	0.50	0.65	1880
1	0.06	0.53	0.11	120
accuracy			0.50	2000
macro avg	0.50	0.51	0.38	2000
weighted avg	0.89	0.50	0.62	2000

Overall Accuracy: 0.5030

Precision (for Verified=1): 0.0630

Recall (for Verified=1): 0.5250

F1-Score (for Verified=1): 0.1125

Interpretation of Results:

The Confusion Matrix shows the number of true positives, true negatives,
- **Precision** indicates how many of the predicted 'verified' users were
- **Recall** indicates how many of the actual 'verified' users were corre
- **F1-Score** is the harmonic mean of precision and recall, providing a

✓ 4. Model Training

We will now initialize and train a `LogisticRegression` model from `scikit-learn`.
This model is suitable for binary classification tasks, like predicting `verified_status`.

```
# Initialize the Logistic Regression model
# Set max_iter for convergence and solver that handles L2 regularization
model = LogisticRegression(random_state=42, solver='liblinear', max_iter=1000)

# Train the model using the resampled training data
model.fit(X_resampled, y_resampled)

print("Logistic Regression model trained successfully on resampled data.")

Logistic Regression model trained successfully on resampled data.
```

✓ Predicting Statutory Fines for Corporate Regulatory Violations

This notebook demonstrates a complete workflow for building a linear regression model to predict statutory fines for corporate regulatory violations. We will cover data simulation, exploratory data analysis, preprocessing, model training, assumption checking, and evaluation.

✓ 1. Data Simulation

Since we don't have an external dataset, we'll create a synthetic dataset using `pandas` and `numpy`. This dataset will include features such as 'Violation_Severity', 'Company_Size', 'Prior_Violations', 'Region_Code', and our target variable 'Fine_Amount'. This simulation helps us demonstrate the full machine learning pipeline.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_
import statsmodels.api as sm

# Set a random seed for reproducibility
np.random.seed(42)

# Number of rows in the dataset
n_samples = 1000

# 1. Violation_Severity: (1 to 5)
violation_severity = np.random.randint(1, 6, n_samples)

# 2. Company_Size: (number of employees, e.g., 50 to 5000)
company_size = np.random.randint(50, 5001, n_samples)

# 3. Prior_Violations: (count, e.g., 0 to 10)
prior_violations = np.random.randint(0, 11, n_samples)

# 4. Region_Code: (Categorical: North, South, East, West)
region_codes = np.random.choice(['North', 'South', 'East', 'West'], n_sa
```

```

# Base Fine_Amount - designed to have a linear relationship with feature
# Let's create a dependency: higher severity, larger company, more prior
# Region_Code can also have an impact.

# Coefficients for the linear relationship
beta_severity = 5000
beta_company_size = 0.5
beta_prior_violations = 1000

# Region effects (e.g., North is baseline, South +5000, East +2000, West
region_effect = np.zeros(n_samples)
region_effect[region_codes == 'South'] = 5000
region_effect[region_codes == 'East'] = 2000
region_effect[region_codes == 'West'] = 7000

# Calculate base fine amount
base_fine_amount = (
    beta_severity * violation_severity +
    beta_company_size * company_size +
    beta_prior_violations * prior_violations +
    region_effect
)

# Add random noise (e.g., normal distribution with some standard deviat
noise = np.random.normal(0, 2000, n_samples) # Standard deviation of $20

# Ensure Fine_Amount is always positive
fine_amount = np.maximum(500, base_fine_amount + noise)

# Create the DataFrame
df = pd.DataFrame({
    'Violation_Severity': violation_severity,
    'Company_Size': company_size,
    'Prior_Violations': prior_violations,
    'Region_Code': region_codes,
    'Fine_Amount': fine_amount
})

# Display the first 5 rows of the simulated dataset
print("First 5 rows of the simulated dataset:")
display(df.head())

# Display basic information about the dataset
print("\nInformation about the simulated dataset:")
display(df.info())

```

First 5 rows of the simulated dataset:

	Violation_Severity	Company_Size	Prior_Violations	Region_Code	Fine_A
0	4	3581	0	South	24102.2
1	5	4501	1	North	25043.9
2	3	2985	4	East	22987.5
3	5	3926	10	West	44263.8
4	5	176	4	South	34289.0

Information about the simulated dataset:

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 1000 entries, 0 to 999

Data columns (total 5 columns):

#	Column	Non-Null Count	Dtype
0	Violation_Severity	1000 non-null	int64
1	Company_Size	1000 non-null	int64
2	Prior_Violations	1000 non-null	int64
3	Region_Code	1000 non-null	object
4	Fine_Amount	1000 non-null	float64

dtypes: float64(1), int64(3), object(1)

memory usage: 39.2+ KB

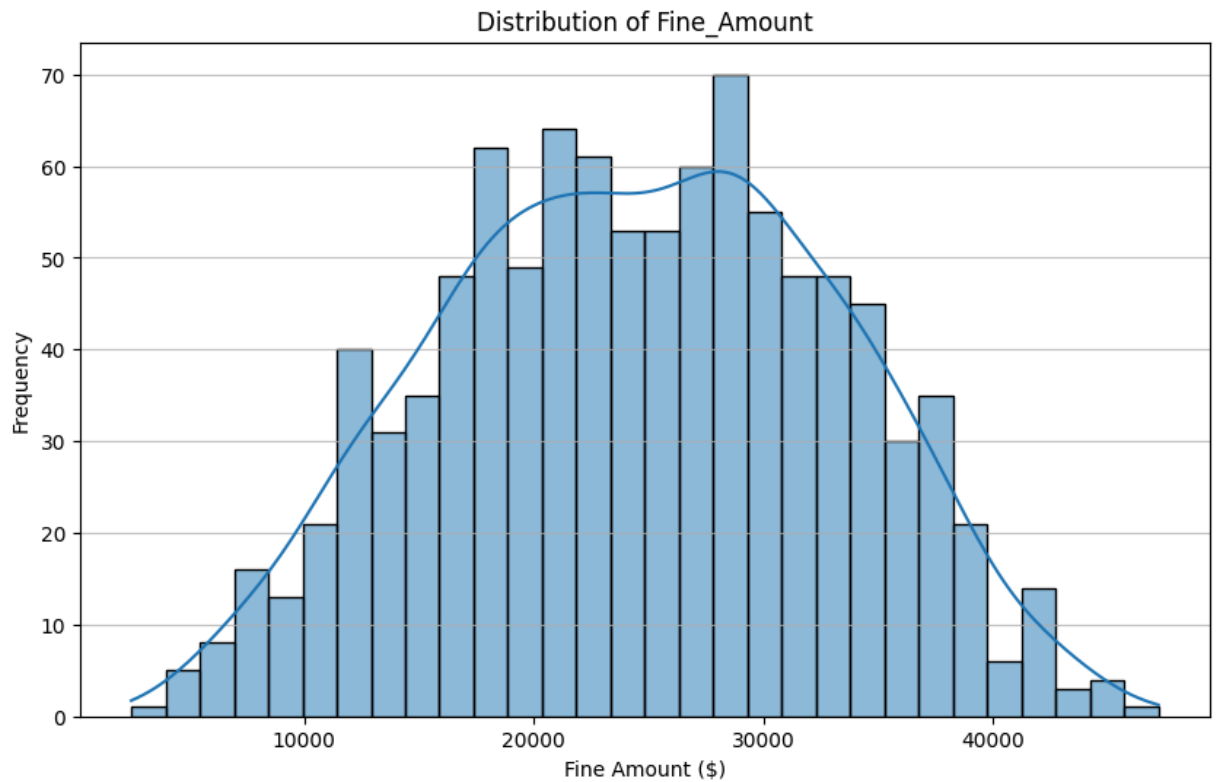
None

✓ 2. Exploratory Data Analysis (EDA)

Exploratory Data Analysis helps us understand the distribution of our data, identify relationships between variables, and detect potential patterns or anomalies. We will use `matplotlib` and `seaborn` to visualize key aspects of our dataset.

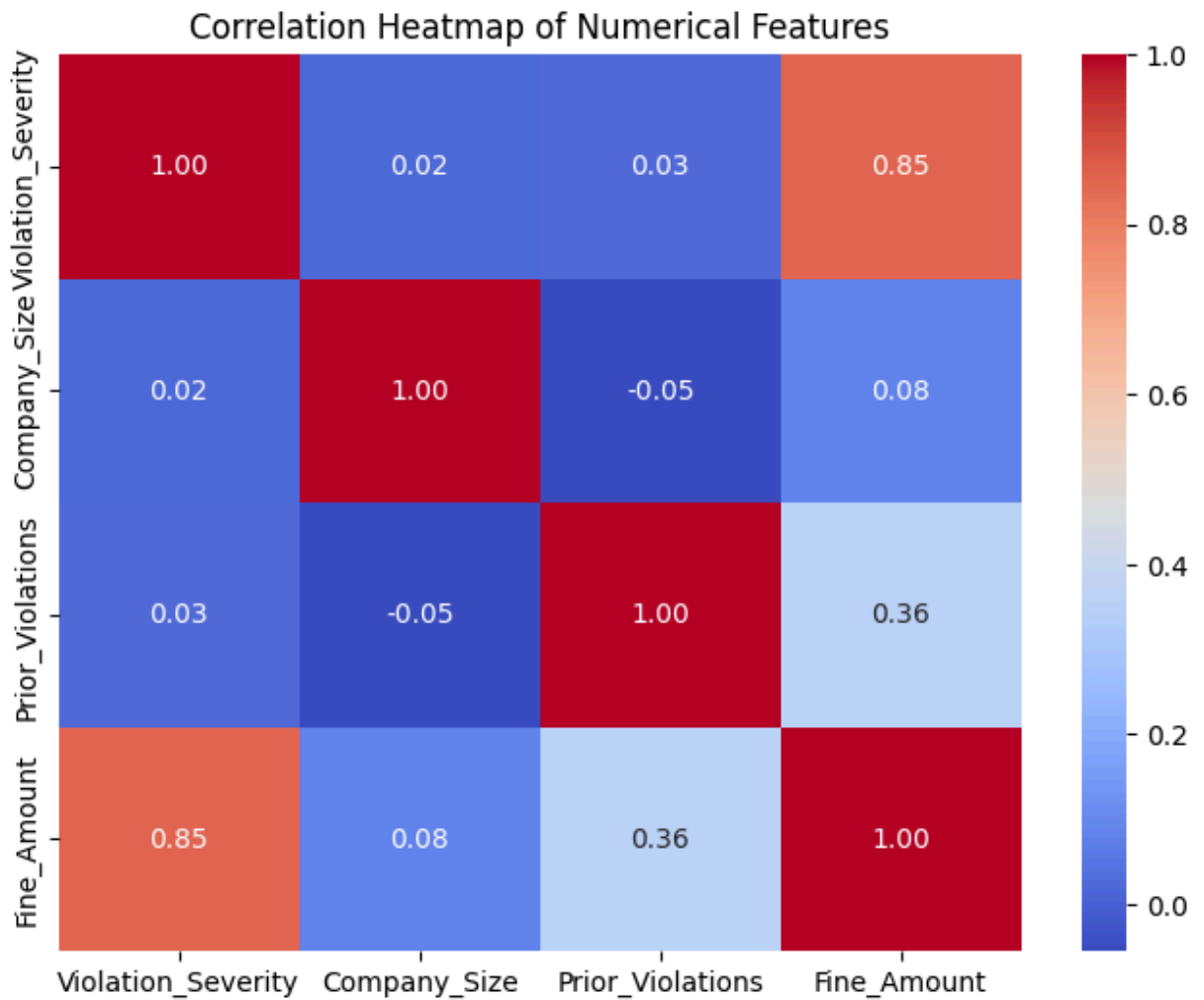
✓ Distribution of 'Fine_Amount'

```
plt.figure(figsize=(10, 6))
sns.histplot(df['Fine_Amount'], bins=30, kde=True)
plt.title('Distribution of Fine_Amount')
plt.xlabel('Fine Amount ($)')
plt.ylabel('Frequency')
plt.grid(axis='y', alpha=0.75)
plt.show()
```



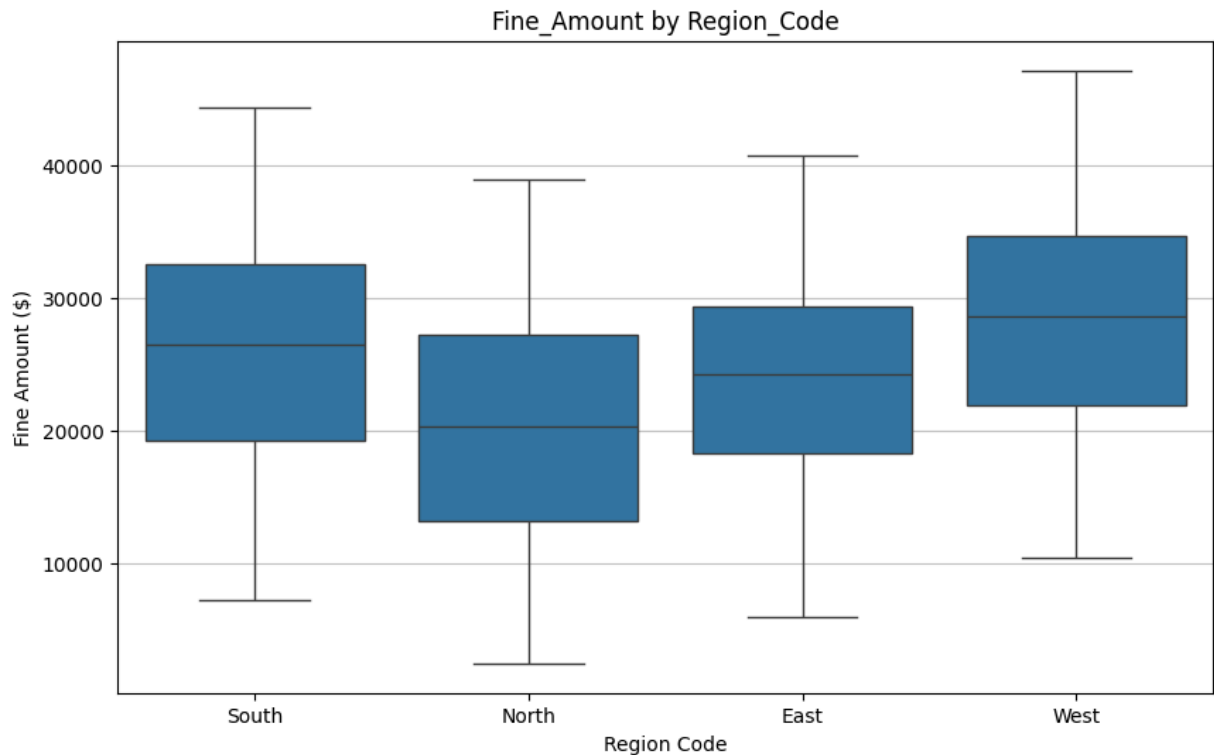
✓ Correlation Heatmap of Numerical Features

```
numerical_df = df.select_dtypes(include=np.number)
plt.figure(figsize=(8, 6))
sns.heatmap(numerical_df.corr(), annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Correlation Heatmap of Numerical Features')
plt.show()
```



✓ Boxplot of 'Fine_Amount' Across Different 'Region_Codes'

```
plt.figure(figsize=(10, 6))
sns.boxplot(x='Region_Code', y='Fine_Amount', data=df)
plt.title('Fine_Amount by Region_Code')
plt.xlabel('Region Code')
plt.ylabel('Fine Amount ($)')
plt.grid(axis='y', alpha=0.75)
plt.show()
```



✓ 3. Preprocessing

Data preprocessing is crucial to prepare our raw data for machine learning models. This involves handling categorical variables using one-hot encoding, scaling numerical features to ensure they contribute equally to the model, and splitting the data into training and testing sets.

✓ One-Hot Encoding for Categorical Variables

```
# Apply One-Hot Encoding to 'Region_Code'
df_encoded = pd.get_dummies(df, columns=['Region_Code'], drop_first=True)

print("DataFrame after One-Hot Encoding:")
display(df_encoded.head())
```

DataFrame after One-Hot Encoding:

	Violation_Severity	Company_Size	Prior_Violations	Fine_Amount	Region
0	4	3581	0	24102.271059	
1	5	4501	1	25043.979066	
2	3	2985	4	22987.523833	
3	5	3926	10	44263.810876	
4	5	176	4	34289.040189	

✓ Splitting Data into Features (X) and Target (y)

```
X = df_encoded.drop('Fine_Amount', axis=1)
y = df_encoded['Fine_Amount']
```

```
print("Features (X) head:")
display(X.head())
print("\nTarget (y) head:")
display(y.head())
```

Features (X) head:

	Violation_Severity	Company_Size	Prior_Violations	Region_Code_North
0	4	3581	0	False
1	5	4501	1	True
2	3	2985	4	False
3	5	3926	10	False
4	5	176	4	False

Target (y) head:

	Fine_Amount
0	24102.271059
1	25043.979066
2	22987.523833
3	44263.810876
4	34289.040189

dtype: float64

Scaling Numerical Features

```
# Identify numerical columns for scaling (excluding one-hot encoded columns)
numerical_cols = ['Violation_Severity', 'Company_Size', 'Prior_Violation']

scaler = StandardScaler()
X[numerical_cols] = scaler.fit_transform(X[numerical_cols])

print("Features (X) after scaling numerical columns:")
display(X.head())
```

Features (X) after scaling numerical columns:

	Violation_Severity	Company_Size	Prior_Violations	Region_Code_North
0	0.695300	0.725423	-1.593608	False
1	1.393393	1.365150	-1.265435	True
2	-0.002792	0.310991	-0.280916	False
3	1.393393	0.965320	1.688122	False
4	1.393393	-1.642261	-0.280916	False

Train-Test Split (80/20)

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)

print(f"X_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"y_train shape: {y_train.shape}")
print(f"y_test shape: {y_test.shape}")
```

```
X_train shape: (800, 6)
X_test shape: (200, 6)
y_train shape: (800,)
y_test shape: (200,)
```

4. Model Training

We will now initialize and train a `LinearRegression` model from `scikit-learn`. The model learns the linear relationship between our preprocessed features and the target 'Fine_Amount'.

```
# Initialize the Linear Regression model
model = LinearRegression()

# Train the model using the training data
model.fit(X_train, y_train)

print("Linear Regression model trained successfully.")
```

```
Linear Regression model trained successfully.
```

✓ 5. Assumption Checking

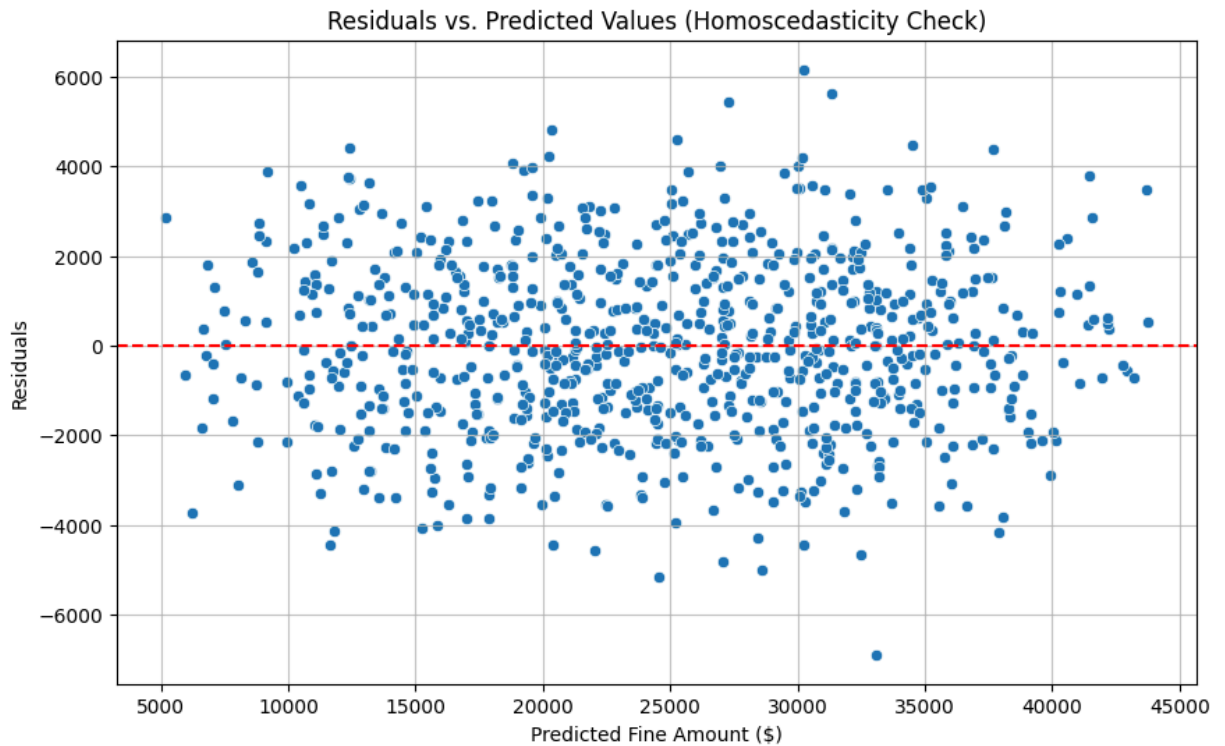
Linear regression relies on several assumptions, such as linearity, independence of errors, homoscedasticity, and normality of residuals. We will visually inspect the residuals to check for homoscedasticity (constant variance of errors) and normality.

✓ Residual Plot (Residuals vs. Predicted Values) for Homoscedasticity

```
# Make predictions on the training set to get residuals
y_train_pred = model.predict(X_train)
residuals = y_train - y_train_pred

plt.figure(figsize=(10, 6))
sns.scatterplot(x=y_train_pred, y=residuals)
plt.axhline(y=0, color='r', linestyle='--')
plt.title('Residuals vs. Predicted Values (Homoscedasticity Check)')
plt.xlabel('Predicted Fine Amount ($)')
plt.ylabel('Residuals')
plt.grid(alpha=0.75)
plt.show()

print("Ideally, the residuals should be randomly scattered around zero,
```



Ideally, the residuals should be randomly scattered around zero, forming

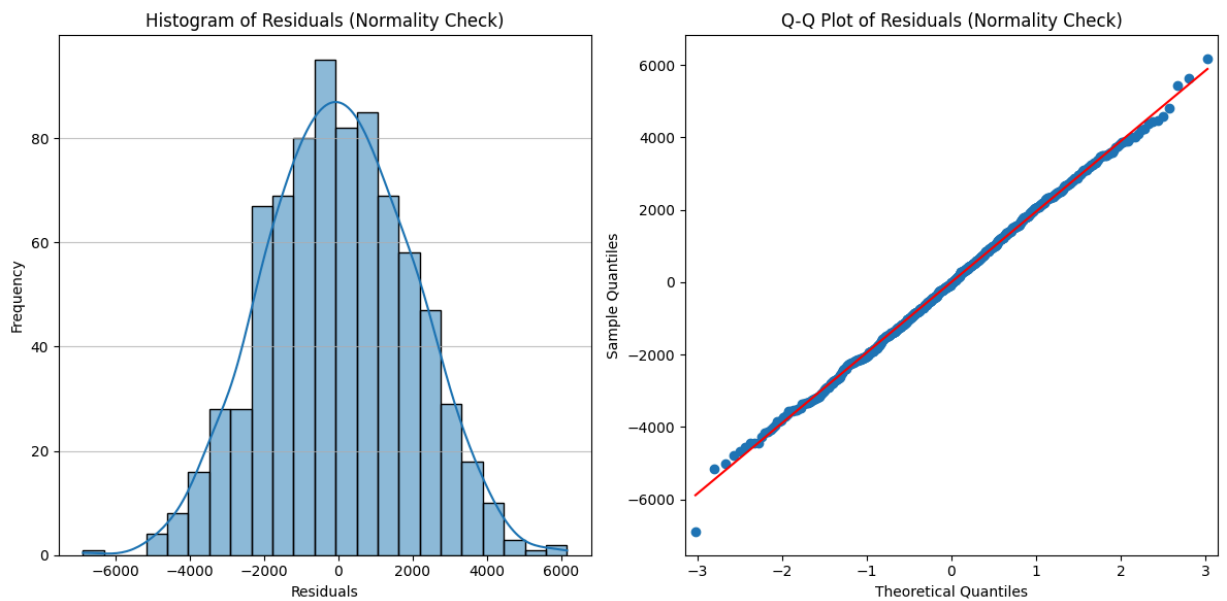
✓ Q-Q Plot or Histogram of Residuals for Normality

```
plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
sns.histplot(residuals, kde=True)
plt.title('Histogram of Residuals (Normality Check)')
plt.xlabel('Residuals')
plt.ylabel('Frequency')
plt.grid(axis='y', alpha=0.75)

plt.subplot(1, 2, 2)
sm.qqplot(residuals, line='s', ax=plt.gca())
plt.title('Q-Q Plot of Residuals (Normality Check)')

plt.tight_layout()
plt.show()
```



For normality, the histogram of residuals should approximate a normal (be

✓ 6. Evaluation

Finally, we evaluate the performance of our trained model on the unseen test data using common regression metrics: R^2 Score, Mean Absolute Error (MAE), and Root Mean Squared Error (RMSE). We will also examine the model's coefficients to understand the influence of each feature on the 'Fine_Amount'.

```
# Make predictions on the test set
y_pred = model.predict(X_test)

# Calculate evaluation metrics
r2 = r2_score(y_test, y_pred)
```